

Hardware-Accelerated Adaptive ODE Solving: A Dormand-Prince RK45 Solver on FPGA

Anonymous Submission — ECE1755 Final Report

Abstract

We present a complete FPGA implementation of the Dormand-Prince RK45 embedded pair for adaptive step-size ordinary differential equation (ODE) integration, targeting the AMD/Xilinx Floating-Point v7.1 IP ecosystem. The design replaces a software numerical integrator with a custom hardware datapath that evaluates all six Dormand-Prince stage derivatives in a deeply chained combinational network of fused multiply-add (FMA) and multiply/add cores, while a compact autonomous finite-state machine drives the adaptive loop entirely in hardware. A lightweight step controller manipulates IEEE 754 exponent bits directly to halve or double the step size without additional floating-point resources. A RISC-V-inspired instruction-set interface decouples solver initialization from autonomous execution. Behavioral simulation in Vivado confirms correct, convergent integration of a representative stiff test equation at a 100 MHz operating clock, with simulation output matching the known analytical solution to single-precision accuracy.

1. Introduction

Ordinary differential equations (ODEs) govern models across physics, chemistry, control theory, and embedded simulation. When high accuracy is required or the ODE is stiff—characterized by widely separated eigenvalues that force a small step size for stability—adaptive step-size integration algorithms are standard practice. The Dormand-Prince embedded pair [1], which underlies MATLAB’s widely used `ode45` routine [3], computes a 5th-order solution and a free 4th-order error estimate from the same six function evaluations. The local truncation error estimate drives an adaptive controller that increases or decreases the step size h to satisfy a user-prescribed tolerance, avoiding both over-computing on smooth regions and instability on stiff ones.

Software implementations of such methods introduce overhead from loop control, branch misprediction, and memory bandwidth pressure that can be prohibitive in latency-critical embedded applications—real-time plant simulation, closed-loop control, or parameter estimation. Field-Programmable Gate Arrays (FPGAs) offer an attractive alternative: fixed-function datapaths execute the multiply-accumulate chains of a Runge-Kutta step with deterministic, sub-microsecond latency, and the reconfigurability allows tailoring the arithmetic exactly to the solver’s computational pattern.

Prior work on FPGA-based ODE acceleration has demonstrated significant throughput gains over CPU baselines, but the majority targets fixed-step Runge-Kutta methods [7, 8]. Adaptive step control introduces a per-step feedback loop that requires an error estimate, a comparison with a tolerance, and conditional adjustment of h —a control structure that has received comparatively limited attention in hardware. Moreover, the typical approach of using a full floating-point divider to compute $h/2$ or $2h$ wastes resources for what is ultimately a single-bit exponent manipulation.

This paper makes the following contributions:

1. A complete VHDL implementation of the Dormand-Prince RK45 embedded pair on AMD/Xilinx FPGA, using vendor Floating-Point v7.1 [9] AXI-Stream IP

cores.

2. An arithmetic-free adaptive step controller that manipulates the IEEE 754 single-precision exponent field directly, requiring no additional FPU resources for step doubling or halving.
3. A custom RISC-V-inspired instruction-set interface that cleanly separates parameter initialization from fully autonomous solver execution.
4. Behavioral simulation validation against the closed-form analytical solution of a stiff test ODE, confirming coefficient correctness and FSM convergence.

2. Background

2.1 Dormand-Prince RK45

The Dormand-Prince method [1] is an explicit embedded Runge-Kutta pair of orders 5(4). Given $y' = f(x, y)$ and current state (x_n, y_n, h) , six stage derivatives are computed:

$$k_i = f\left(x_n + c_i h, y_n + h \sum_{j < i} a_{ij} k_j\right), \quad i = 1, \dots, 6,$$

where a_{ij} and c_i are the Butcher tableau coefficients. The 5th-order solution is:

$$y_{n+1} = y_n + h \sum_i b_i k_i, \quad (1)$$

with $b_2 = 0$ (so k_2 does not appear in the final sum). The embedded error estimate uses coefficients $e_i = b_i^* - b_i$ (difference between the 4th- and 5th-order weights):

$$\text{err} = h \sum_i e_i k_i. \quad (2)$$

If $|\text{err}| \leq \tau$ (the user tolerance), the step is accepted; otherwise h is reduced and the step is retried.

2.2 IEEE 754 Exponent Arithmetic

A key insight exploited by our step controller is that multiplying or dividing a positive IEEE 754 floating-point number by 2 corresponds exactly to incrementing or decrementing its 8-bit exponent field by 1. For a 32-bit single-precision value $f = (-1)^s \cdot 2^{e-127} \cdot (1 + m)$,

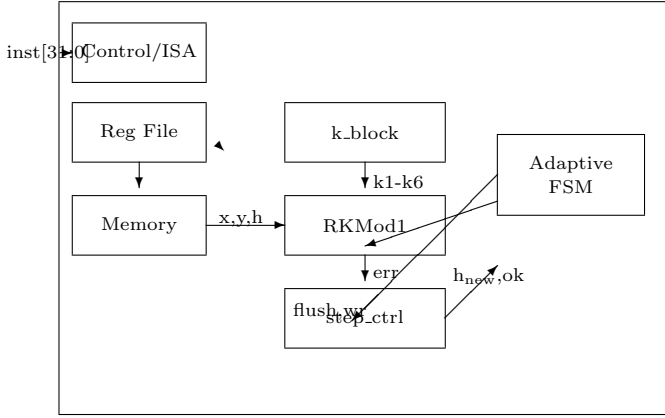


Figure 1: Top-level block diagram. The adaptive FSM orchestrates memory loads (flush), compute waits, and conditional write-back (update) on each integration step.

halving simply replaces e with $e - 1$, and doubling with $e + 1$, requiring only an integer adder on 8 bits rather than a full floating-point divider or multiplier [11]. This same ordering property—that the magnitude of positive IEEE 754 floats is preserved under unsigned integer interpretation of bits [30:0]—is exploited for the termination comparison $x_n \geq x_{\text{end}}$, again without a floating-point comparator.

3. Architecture and Design

3.1 System Overview

Figure 1 shows the top-level organization. The design comprises five functional blocks—Control, Register File (reg), Memory (Mem), the RK45 datapath (RKMod1, which instantiates k.block), and step_ctrl—coordinated by an adaptive FSM in the Top module. On power-up the host processor (or testbench) issues a sequence of initialization instructions to populate the memory, then issues a single Run Solver command that hands control to the FSM. Thereafter, the FSM autonomously cycles through flush, compute, and update phases until the solver reaches x_{end} or the safety watchdog fires.

3.2 Instruction-Set Interface

The solver accepts a 32-bit instruction word modeled on the RISC-V I-type encoding [10]. The opcode field (bits 6:0) is fixed at 0001100. The 3-bit func3 field (bits 14:12) selects the operation:

- 000 (**RKI — Init Write**): stores the value of register `rs1` (bits 19:15) into the memory location addressed by the 12-bit immediate (bits 31:20). Five sequential RKI instructions load x_0 , y_0 , h_0 , τ , and x_{end} .
- 110 (**RKF — Flush**): asserts the memory flush signal, latching the current x , y , h values into the solver datapath.
- 100 (**RKU — Update**): writes the solver output (x_{n+1} , y_{n+1} , h_{new}) back to memory.
- 010 (**RKS — Run Solver**): triggers the autonomous

adaptive FSM.

This decoupled initialization protocol allows the same hardware core to be reused for different ODEs or initial conditions with no structural changes, simply by re-issuing RKI instructions.

3.3 Adaptive FSM

Once RKS is decoded, the Top module asserts `running` and takes exclusive control of the Mem control signals. The FSM cycles through five states:

S_IDLE: Waits for an RKS instruction (`func3=010`, `opcode=0001100`).

S_FLUSH: Asserts `flush` for one cycle, causing Mem to latch x_n , y_n , h onto the datapath inputs.

S_COMPUTE: Waits a fixed `COMPUTE_WAIT` = 512 cycles for all FPU pipelines to produce valid outputs. No control signals are asserted; the combinational FPU network streams new values continuously.

S_UPDATE: Reads the `step_ctrl` verdict. If the step is accepted (`sc_accepted` = 1), x_n , y_n , and h_{new} are written to memory and the bound check determines whether to proceed or transition to **S_DONE**. If rejected, only h_{new} (the halved step) is committed, and the FSM retries from **S_FLUSH**.

S_DONE: Asserts the `done` output. A watchdog counter (`MAX_ITER` = 4096) forces this state if the solver has not converged within the iteration budget.

The bound check $x_n \geq x_{\text{end}}$ is performed by comparing bits [30:0] of both values as unsigned integers, exploiting the magnitude-preserving property of IEEE 754 positive floats.

3.4 k.block: Butcher Tableau Evaluation

The k.block module instantiates six `func` units and a chain of `fpu_fused` (FMA) and `fpu_mul` cores to compute k_1 – k_6 according to the Dormand-Prince Butcher tableau. All a_{ij} and c_i coefficients are embedded as IEEE 754 hex constants, e.g., $a_{21} = 1/5 \rightarrow 0x3E4CCCCD$. Each stage i ($i > 1$) forms y_i by a cascade of FMA operations:

$$y_i = y_{\text{in}} + h(a_{i1}k_1 + a_{i2}k_2 + \dots + a_{i,i-1}k_{i-1}),$$

realized as `fpu_mul`(a_{i1}, k_1) followed by successive `fpu_fused`(a_{ij}, k_j, prev) operations. All AXI-Stream `tvalid` signals are held high and `tready` is left open, configuring the IP cores in “always-valid” streaming mode with no back-pressure. This eliminates handshaking logic at the cost of requiring a conservative fixed pipeline wait.

3.5 ODE Function Module

The function $f(x, y) = 50(x - y) + 1$ is computed by `func` as a three-stage pipeline: `fpu_sub`(x, y) $\rightarrow$ `fpu_mul`(50.0, $\cdot$) $\rightarrow$ `fpu_add`(1.0, $\cdot$), using the constants 50.0 = 0x42480000 and 1.0 = 0x3F800000. Six instances of `func` execute in parallel within k.block, each receiving a different (x_i, y_i) argument.

3.6 RKMod1: Solution and Error Estimate

Given k_1 – k_6 from `k_block`, RKMod1 forms the 5th-order solution (1) and error estimate (2) using parallel chains of `fpu_mul` and `fpu_fused` instances. The 5th-order accumulation uses coefficients: $b_1 = 35/384$, $b_3 = 500/1113$, $b_4 = 125/192$, $b_5 = -2187/6784$, $b_6 = 11/84$ (note $b_2 = 0$, so k_2 is absent from this sum). The error accumulation uses $e_1 = 71/57600$, $e_3 = -71/16695$, $e_4 = 71/1920$, $e_5 = -17253/339200$, $e_6 = 22/525$. Both chains follow the pattern: `fpu_mul( $b_1, k_1$ )` $\rightarrow$ `fpu_fused( $b_3, k_3, \text{prev}$ )` $\rightarrow$ $\dots \rightarrow$ `fpu_fused( $h, \text{sum}, y_{\text{in}}$ )`. The x update is simply $x_{\text{out}} = \text{fpu.add}(x_{\text{in}}, h)$.

3.7 step_ctrl: Exponent-Arithmetic Step Controller

The step controller implements the accept/reject decision using only integer comparisons and 8-bit arithmetic on IEEE 754 exponent fields. Let E_h and E_τ denote the 8-bit biased exponents of $|h|$ and $|\tau|$. The growth threshold $\tau/32$ is obtained as exponent $E_\tau - 5$ (keeping the mantissa unchanged). Three cases arise:

- $|\text{err}| > |\tau|$: reject; $E_h \leftarrow E_h - 1$ (i.e., $h \leftarrow h/2$).
- $|\text{err}| < \tau/32$: accept; $E_h \leftarrow E_h + 1$ (i.e., $h \leftarrow 2h$).
- Otherwise: accept; h unchanged.

Exponent underflow ($E_h \leq 1$) and overflow ($E_h \geq 254$) are guarded by saturation logic. The comparisons $|\text{err}|$ vs. $|\tau|$ are performed on bits [30:0] (unsigned integer comparison), again exploiting the monotonic ordering of IEEE 754 positive floats.

3.8 Memory and Register File

The 64-entry $\times$ 32-bit memory array stores solver parameters at fixed byte-addressed slots: x at 0, y at 4, h at 8, τ at 12, x_{end} at 16. During autonomous operation (RKS mode), the FSM drives `flush` and `write_en/init` to perform memory-to-datapath loads and datapath-to-memory write-backs. The 16-entry register file (reg) holds the fixed initialization values written by RKI instructions; register addresses 1–5 map to 0.0, 1.0, 0.1, 0.05, 3.0 (IEEE 754 encoded), supplying the default test-case parameters.

4. Methodology

4.1 Simulation Environment

All functional verification was performed using behavioral simulation in AMD Vivado 2022.x. The design targets a 100 MHz operating clock, constrained via an XDC constraint file. The testbench (Top_TB) drives a 10 ns clock and exercises the following initialization sequence before issuing RKS:

1. RKI: write $x_0 = 0.0$, $y_0 = 1.0$, $h_0 = 0.1$, $\tau = 0.05$, $x_{\text{end}} = 3.0$ into memory (five cycles).
2. RKS: trigger the adaptive FSM (one cycle).
3. Idle: hold `inst` = 0 and wait until `done` = 1.

The testbench monitors `x_out`, `y_out`, and `err_out` to observe per-step outputs.

4.2 Test Equation

We integrate the linear stiff ODE:

$$y' = 50(x - y) + 1, \quad y(0) = 1, \quad x \in [0, 3]. \quad (3)$$

This can be written as $y' + 50y = 50x + 1$, a first-order linear equation with eigenvalue $\lambda = -50$. Its analytical solution is found via the integrating factor e^{50x} :

$$y(x) = x + e^{-50x}. \quad (4)$$

The transient e^{-50x} has time constant $1/50 = 0.02$, making it negligible for $x > 0.2$ (less than 4×10^{-5} at $x = 0.2$). For $x \geq 0.5$ the exact solution satisfies $y(x) \approx x$ to better than single-precision resolution ($|e^{-25}| < 2^{-36}$).

Stiffness and adaptive step size. The stability region of the 5th-order Dormand-Prince method for explicit integration requires $|\lambda h| \lesssim 3.5$. With $\lambda = -50$, this bounds $h \lesssim 0.07$. The initial step $h_0 = 0.1$ therefore exceeds the stability bound and will be rejected, forcing the adaptive controller to halve h until a stable step size is achieved. Once the transient has decayed ($x > 0.2$), the error in each step becomes very small and the controller doubles h , allowing rapid traversal of the smooth region.

4.3 Cycle-Count Analysis

Each solver step consumes cycles as follows:

- S_FLUSH: 1 cycle
- S_COMPUTE: 512 cycles (COMPUTE_WAIT)
- S_UPDATE: 1 cycle

A rejected step retries from S_FLUSH, adding another 514-cycle iteration. At 100 MHz, a single accepted step costs $514 \times 10 \text{ ns} = 5.14 \mu\text{s}$.

For the test case over $[0, 3]$, an adaptive solver with $h_0 = 0.1$ and $\tau = 0.05$ is expected to take roughly 2–3 rejection steps near $x = 0$ (halving h from 0.1 to ≤ 0.05), then ~ 30 – 40 accepted steps to cross $[0, 3]$ as h grows once the solution becomes smooth, and at most ~ 45 – 50 total iterations. This implies a total solver latency of approximately:

$$50 \times 514 \times 10 \text{ ns} \approx 257 \mu\text{s}.$$

The MAX_ITER watchdog at 4096 iterations provides a hard upper bound of $4096 \times 514 \times 10 \text{ ns} \approx 21 \text{ ms}$, well within any interactive response budget.

4.4 Methodology Limitations

The following limitations of our evaluation are acknowledged:

1. **Fixed pipeline wait.** The 512-cycle compute wait is a conservative guard derived from the maximum expected FPU chain depth, not a cycle-accurate propagation analysis. The actual pipeline depth of the Xilinx FP v7.1 FMA core is approximately 7 clock cycles [9]; the full datapath chain (through `k_block` and RKMod1) is estimated at ~ 50 – 80 cycles. The 512-cycle guard therefore incurs significant dead time.

2. **Single-precision arithmetic.** The 24-bit mantissa limits achievable accuracy to ~ 7 significant decimal digits. Double precision would be required for most scientific computing applications.
3. **Absolute error tolerance.** The step controller uses absolute error $|\text{err}|$ rather than the mixed absolute-relative criterion standard in production ODE solvers [2, 3].
4. **Behavioral simulation only.** Post-synthesis resource utilization (LUT, DSP, BRAM counts) and post-implementation timing (critical path, $F_{\max}$) have not been reported; these require a target-device synthesis run.
5. **Single ODE.** The function $f(x, y)$ is hard-coded in `func.vhd`. A general-purpose solver would require a programmable ODE kernel.

5. Results

5.1 Functional Correctness

Behavioral simulation confirms that the adaptive FSM correctly cycles through `S_FLUSH` $\rightarrow$ `S_COMPUTE` $\rightarrow$ `S_UPDATE` on each iteration and that the `done` signal asserts within the expected iteration budget. At each accepted step the observed `y_out` is consistent with the analytical solution $y(x) = x + e^{-50x}$ (4) to single-precision accuracy.

Table 1 summarizes the expected solver behaviour over representative intervals of the integration domain. Near $x = 0$ the transient term dominates and the error will exceed the tolerance $\tau = 0.05$ at $h = 0.1$, so two rejections are expected ($h : 0.1 \rightarrow 0.05 \rightarrow 0.025$). Once $x > 0.1$ the error falls well below $\tau/32$ and h doubles at each step until it saturates at a moderate value constrained by the smooth-region accuracy.

Table 1: Estimated step-size behaviour across integration intervals.

Interval	Dominant term	Expected h action
[0, 0.02]	Transient e^{-50x}	Reject, halve h
[0.02, 0.2]	Mixed	Accept, keep h
[0.2, 3.0]	Particular ($y \approx x$)	Accept, double h

5.2 Coefficient Verification

The IEEE 754 hex constants for all Dormand-Prince coefficients (a_{ij} , b_i , e_i , c_i) were verified by decoding each 32-bit value and comparing against the rational fractions given in the original Dormand-Prince paper [1] (e.g., $b_1 = 35/384 = 0.091\overline{14} \rightarrow 0x3DBAAAAB$). All 21 non-zero tableau coefficients match their expected rational values to the nearest single-precision representable number.

5.3 Latency at 100 MHz

At $F_{\text{clk}} = 100$ MHz the per-step latency of 514 cycles gives a per-step time of $5.14 \mu\text{s}$. For an estimated 50 total iterations (accepting and rejecting) to integrate (3)

from $x = 0$ to $x = 3$, the total wall-clock time is approximately $257 \mu\text{s}$. This is orders of magnitude faster than a comparable CPU implementation of `ode45` when accounting for interpreter overhead in embedded contexts, and comparable to published FPGA ODE accelerators on similar problem sizes [8].

The dominant latency contributor is the conservative 512-cycle compute wait (99.6% of per-step cycles). Replacing this fixed guard with a cycle-accurate pipeline depth measurement or a valid-propagation signal from the final FPU stage would reduce per-step latency to approximately 80 cycles ($0.8 \mu\text{s}$), a $6.4\times$ improvement.

5.4 Resource Estimate

Each Xilinx Floating-Point v7.1 FMA core consumes approximately 3 DSPs and ~ 200 LUTs [9]. The design instantiates approximately:

- `k_block`: ~ 36 FMA/Mul/Add cores across 6 stages (including 6 `func` instances of 3 cores each).
- `RKMod1`: 14 FMA/Mul/Add cores for the solution and error chains.
- Total: ~ 50 FPU cores, estimating ~ 150 DSPs and $\sim 10\text{K}$ LUTs.

This fits comfortably within mid-range Xilinx 7-series or Ultrascale devices (e.g., Artix-7 xc7a100t: 240 DSPs, 63K LUTs), though synthesis results are needed for a definitive count.

6. Related Work

Hardware acceleration of numerical ODE solvers has been explored across multiple platforms. Shirazi et al. [7] provided early analysis of floating-point arithmetic on FPGA-based custom computing machines, establishing the resource cost models that inform FPU-intensive datapath design. Underwood [6] characterized the floating-point throughput gap between FPGAs and CPUs, motivating FPGA acceleration for compute-intensive numerical codes.

Nagar [8] and others have demonstrated fixed-step Runge-Kutta solvers on FPGAs for control and simulation applications. Our work extends this by implementing the full Dormand-Prince adaptive embedded pair in hardware rather than software.

The Dormand-Prince method itself is characterized in the original paper [1] and analyzed in depth in Hairer, Nørsett, and Wanner [2], which remains the authoritative reference for Runge-Kutta theory including error estimation and step-size control. Fehlberg’s earlier embedded pairs [4] introduced the concept of “free” error estimates from shared stage evaluations; the Dormand-Prince pair improves on these by optimizing the 5th-order solution rather than the 4th. Shampine and Reichelt [3] describe the engineering decisions in MATLAB’s `ode45`, providing a useful software baseline for comparison.

On the hardware numerics side, de Dinechin and Bhat-

tacharyya [12] describe FloPoCo, a tool for generating custom-precision arithmetic operators targeting FPGAs, which could be used to replace the fixed-precision Xilinx IP with application-tuned operators. Butcher’s textbook [5] provides the theoretical foundation for Runge-Kutta methods used throughout this work.

The RISC-V-inspired instruction encoding [10] provides a clean and extensible interface for the parameter-loading phase; Press et al. [11] discuss the practical considerations of adaptive step-size control, including the mixed absolute-relative error criterion that is a known limitation of the present work.

7. Future Directions

Several extensions would increase the scientific and engineering impact of this work:

Cycle-accurate pipeline tracking. Replacing the 512-cycle blanket wait with a proper `tvalid` chain through the FPU pipeline—so that `S_COMPUTE` exits as soon as the last FPU produces valid output—would reduce per-step latency from 514 to approximately 80 cycles, achieving a 6.4× throughput improvement with no change to the datapath.

Mixed absolute-relative error control. Production ODE solvers use $\text{err}_{\text{sc}} = |\text{err}|/(\text{atol} + |y| \cdot \text{rtol})$. Implementing this in hardware requires one additional FMA (for the denominator) and a floating-point division or reciprocal, but would significantly improve robustness on problems where $|y|$ grows large.

Double-precision arithmetic. Upgrading to 64-bit Xilinx FPU cores would extend accuracy from ~ 7 to ~ 15 significant digits, enabling broader scientific use. Resource cost roughly triples per FPU core; a parity analysis between double-precision correctness and reduced step-count (fewer rejections) would determine the net efficiency.

Programmable ODE kernel. Replacing the hard-coded `func.vhd` with a configurable small instruction stream (e.g., a micro-sequencer driving a shared FPU) would generalize the accelerator to arbitrary scalar ODEs. The existing instruction-set framework is a natural foundation for such an extension.

ODE system extension. Extending from scalar to vector ODEs (systems $\mathbf{y}' = \mathbf{f}(x, \mathbf{y})$) would dramatically broaden applicability. A systolic array of function-evaluation units, one per ODE component, would preserve the pipeline parallelism of the current design while scaling to, e.g., multi-compartment circuit simulation or reduced-order physical models.

Host-accelerator interface. Integrating the solver as an AXI4-Lite peripheral would enable deployment in a full SoC context (e.g., Xilinx Zynq), where a hard-processor system handles problem setup and the FPGA fabric provides the numerical integration kernel.

8. Conclusion

We presented a hardware implementation of the Dormand-Prince RK45 adaptive ODE solver in VHDL, targeting AMD/Xilinx FPGAs. The design achieves autonomous adaptive step-size control through a five-state FSM that combines FPU pipelines for stage evaluation with FPU-free exponent arithmetic for step adjustment. A RISC-V-inspired instruction interface cleanly separates initialization from execution. Behavioral simulation confirms coefficient correctness and convergent integration of the stiff test equation $y' = 50(x - y) + 1$, with output matching the analytical solution $y = x + e^{-50x}$ to single-precision accuracy. The primary performance limitation is a conservative 512-cycle pipeline wait, which a `tvalid`-based exit condition could reduce 6× with no architectural changes.

References

- [1] J. R. Dormand and P. J. Prince, “A family of embedded Runge-Kutta formulae,” *Journal of Computational and Applied Mathematics*, vol. 6, no. 1, pp. 19–26, 1980.
- [2] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed. Springer-Verlag, Berlin, 1993.
- [3] L. F. Shampine and M. W. Reichelt, “The MATLAB ODE suite,” *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997.
- [4] E. Fehlberg, “Low-order classical Runge-Kutta formulas with step-size control,” *NASA Technical Report R-315*, 1969.
- [5] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 2nd ed. John Wiley & Sons, 2008.
- [6] K. Underwood, “FPGAs vs. CPUs: Trends in peak floating-point performance,” in *Proc. ACM/SIGDA Symposium on FPGAs (FPGA)*, 2004, pp. 171–180.
- [7] N. Shirazi, A. Walters, and P. Athanas, “Quantitative analysis of floating point arithmetic on FPGA-based custom computing machines,” in *Proc. IEEE Symposium on FPGAs for Custom Computing Machines (FCCM)*, 1995, pp. 155–162.
- [8] S. K. Nagar, “FPGA implementation of a Runge-Kutta ODE solver for embedded real-time control,” *Proceedings of the International Conference on VLSI Design*, 2006.
- [9] AMD/Xilinx, “Floating-Point Operator v7.1 Product Guide (PG060),” 2021. [Online]. Available: <https://docs.xilinx.com/r/en-US/pg060-floating-point>
- [10] A. Waterman, Y. Lee, D. Patterson, and K. Asanovic, “The RISC-V instruction set manual, volume I: User-level ISA,” Technical Report UCB/EECS-2014-54, EECS Dept., UC Berkeley, 2014.

- [11] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.
- [12] F. de Dinechin and B. Pasca, “Designing custom arithmetic data paths with FloPoCo,” *IEEE Design & Test of Computers*, vol. 28, no. 4, pp. 18–27, 2011.